

用dplyr函数实现数据变换：：速查表



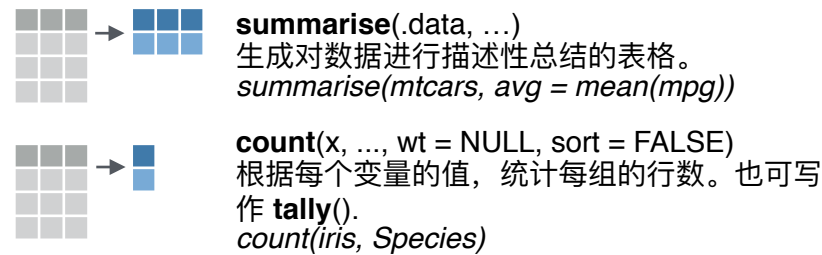
dplyr 函数可应用于“pipes”结构，使用前请保证数据是清洁的，清洁的数据应具有以下特征：



对案例 (cases) 进行描述性总结

这些对数据进行描述性总结的函数作用于列，能够生成对数据进行描述的表格。这些函数输入的是向量 (vector)，返回的是一个值 (见第二页)。

描述总结性函数

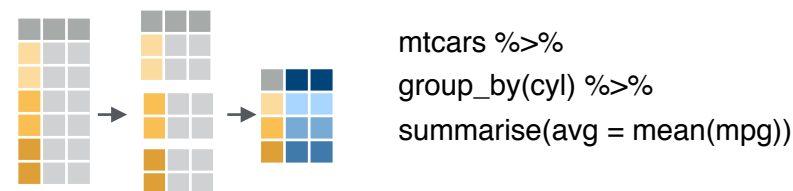


其他形式

summarise_all() - 函数将作用于每一列。
summarise_at() - 函数将作用于特定列。
summarise_if() - 函数将作用于满足条件的所有列。

对案例进行分组

group_by() 函数能够创建一份“被分组”的表格。
dplyr 函数能对每组数据进行分别处理，然后整合结果。



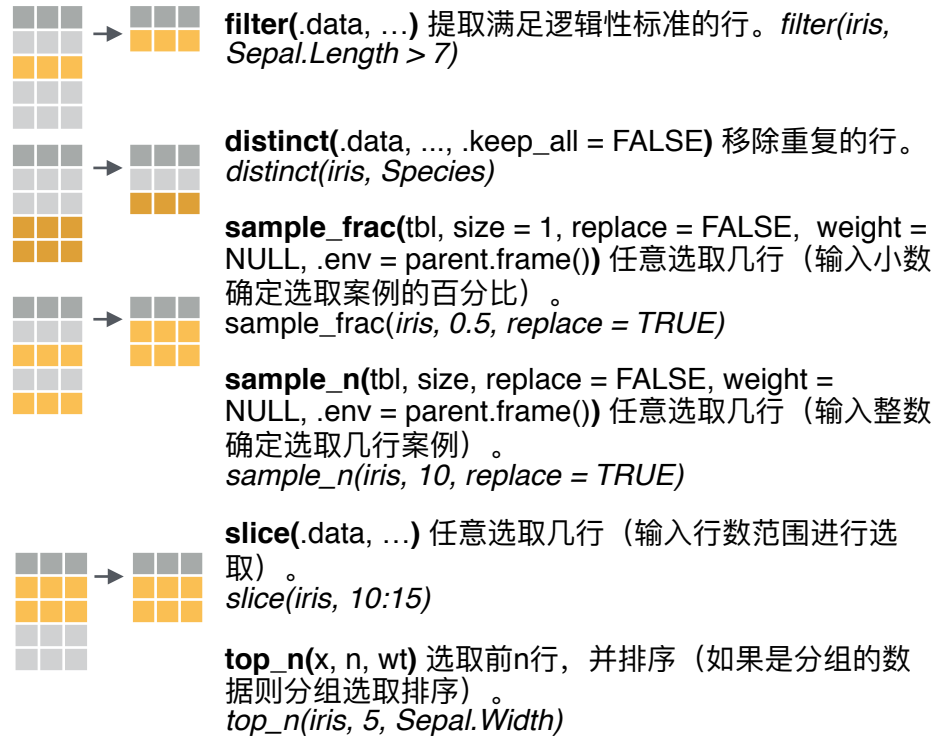
group_by(.data, ..., add = FALSE)
这一函数能够返回一份被分组的表格。
`g_iris <- group_by(iris, Species)`

ungroup(x, ...)
这一函数能够返回一份被取消分组的表格。
`ungroup(g_iris)`

对案例的操作

提取部分案例

这些函数作用于行，能够返回子集，形成新表格。

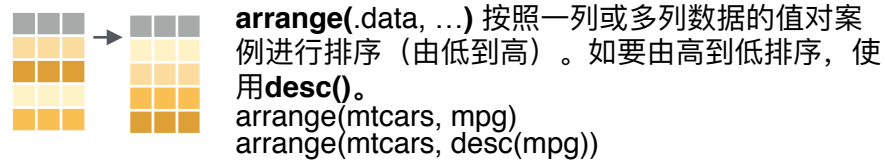


配合函数 filter() 使用的逻辑/布尔数学体系的运算符

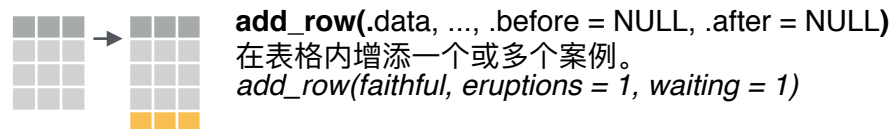
<	<=	is.na()	%in%		xor()
>	>=	!is.na()	!	&	

输入 `?base::logic` 和 `?Comparison` 获取帮助。

为案例排序



增加案例



对变量的操作

提取部分变量

这些函数作用于列，能够返回一组列，形成新的向量或表格。



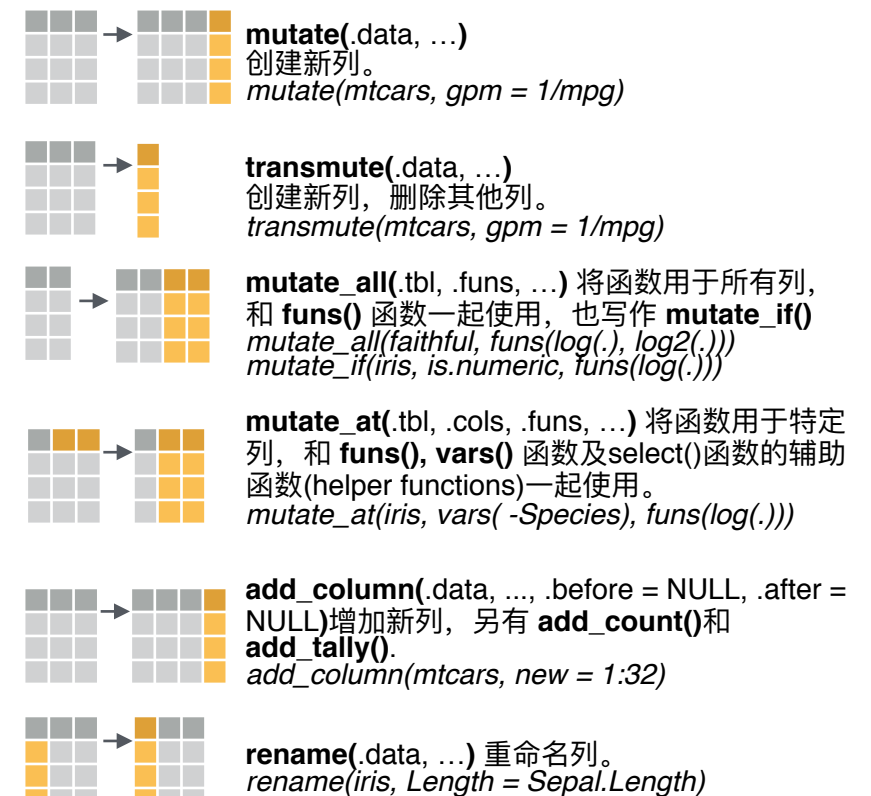
select () 函数的详细用法如下：
e.g. `select(iris, starts_with("Sepal"))`

<code>contains(match)</code>	<code>num_range(prefix, range)</code>	∴ e.g. <code>mpg:cyl</code>
<code>ends_with(match)</code>	<code>one_of(...)</code>	∴ e.g. <code>-Species</code>
<code>matches(match)</code>	<code>starts_with(match)</code>	

创建新变量

这些是作用于列的向量化函数。这些函数输入的是向量，返回的是等长的向量 (见第二页)。

向量化函数



向量函数

与 **MUTATE ()** 一起使用

mutate() 和 **transmute()** 将向量函数作用于列，以创建新的列。向量函数输入的是向量，返回的是与输出等长的向量。

向量函数

数据“偏移”

dplyr::lag() - 将数据向后推一格
dplyr::lead() - 将数据向前推一格

累计计算

dplyr::cumall() - 累计判断逻辑向量是否为“真”
dplyr::cumany() - 累计判断逻辑向量是否有“真”
cummax() - 累计最大值
dplyr::cummean() - 累计平均数
cummin() - 累计最小值
cumprod() - 累计乘积()
cumsum() - 累计和()

排名

dplyr::cume_dist() - <=当前排名之前所有值的比例
dplyr::min_rank() - 同rank(), 默认ties = “min”
dplyr::dense_rank() - 同上, 排名间无间隔
dplyr::ntile() - 分成n份
dplyr::percent_rank() - 同min_rank, 用0~1体现百分比
dplyr::row_number() - 同rank(), 默认ties = “first”

数学运算

+, **-**, *****, **/**, **^**, **%/%**, **%%** - 运算符
log(), **log2()**, **log10()** - 对数
<, **<=**, **>**, **>=**, **!=**, **==** - 逻辑比较
dplyr::between() - 判断x中的值是否在范围内 (通过left, right 设定)
dplyr::near() - safe == 用于浮点数

其他

dplyr::case_when() - 同 if_else(), 但能处理多个案例
dplyr::coalesce() - 寻找第一个非NA的值
dplyr::if_else() - 基于元素的 if() + else()
dplyr::na_if() - 用 NA 取代特定值
pmax() - 基于元素的 max()
pmin() - 基于元素的 min()
dplyr::recode() - 向量版本的 switch()
dplyr::recode_factor() - 向量版本的 switch(), 用于因子

生成描述性数据的函数

与 **SUMMARISE ()** 一起使用

summarise() 对每列数据进行描述性总结，形成一张新表格。总结性函数输入向量，返回单个数值。

描述总结性函数

个数

dplyr::n() - 值/行的个数
dplyr::n_distinct() - # 不重复的值的个数
sum(!is.na()) - # 非NA数值的个数

位置

mean() - 平均数, 或: **mean(!is.na())**
median() - 中位数

逻辑

mean() - 为“真” (TRUE) 的比例
sum() - # 为“真” (TRUE) 的描述性数据

顺序

dplyr::first() - 第一个值
dplyr::last() - 最后一个值
dplyr::nth() - 返回向量中在特定位置的行

排序

quantile() - 分位数
min() - 最小值
max() - 最大值

散布

IQR() - 四分位距
mad() - 绝对中位差
sd() - 标准差
var() - 方差

行名

清洁的数据不使用行名，因其将变量置于列外。如果要编辑行名，需先将其移到一列内。

rownames_to_column()
将行名移到一列内。
`a <- rownames_to_column(iris, var = "C")`

column_to_rownames()
将一列移到行名处。
`column_to_rownames(a, var = "C")`

此外还有: **has_rownames()**, **remove_rownames()**

合并表格

基于变量的合并

x + **y** =

A	B	C	A	B	D
a	t	1	a	t	3
b	u	2	b	u	2
c	v	3	c	v	1

bind_cols() 将两个表格肩并肩放置。

bind_cols(...) 将两个表格肩并肩放置，形成一个表格。
※ 注意行数的一致性。

“变形合并” 基于变量将两个表格合并，并基于值匹配案例。以下是合并两个表格的不同方式。

left_join(x, y, by = NULL, copy=FALSE, suffix=c(".x", ".y"), ...)
将y并入x

right_join(x, y, by = NULL, copy = FALSE, suffix=c(".x", ".y"), ...)
将x并入y

inner_join(x, y, by = NULL, copy = FALSE, suffix=c(".x", ".y"), ...)
合并数据，仅保留匹配的行。

full_join(x, y, by = NULL, copy=FALSE, suffix=c(".x", ".y"), ...)
合并数据，保留所有行和所有列。

用 **by = c("col1", "col2", ...)** 来规定匹配所用的共同列。
`left_join(x, y, by = "A")`

用 **by = c("col1" = "col2")**, 来指明共同列在不同表格中的不同名称。
`left_join(x, y, by = c("C" = "D"))`

用 **suffix = c("1", "2")** 为在不同表格中名称相同却不匹配的列设定后缀。
`left_join(x, y, by = c("C" = "D"), suffix = c("1", "2"))`

基于案例的合并

x + **y** =

A	B	C
a	t	1
b	u	2
c	v	3

bind_rows() 将两个表格上下放置。

bind_rows(..., .id = NULL)
将两个表格上下放置，形成一个表格。可以用.id= 设置一个指明其原表格名称的列 (如图中的DF列)。

intersect(x, y, ...)
在x和y中都出现的行。

setdiff(x, y, ...)
在x中出现，在y中未出现的行。

union(x, y, ...)
在x或y中出现的行 (重复的行删去)。如要保留重复行，使用union_all()。

setequal() 函数能够检查两组数据是否拥有完全一致的行 (顺序不限)。

提取行

x + **y** =

A	B	C	A	B	D
a	t	1	a	t	3
b	u	2	b	u	2
c	v	3	c	v	1

“筛选合并” 能用一组数据筛选另一组数据。

semi_join(x, y, by = NULL, ...)
返回 x 的行中与 y 中的行相同的部分。※ 可用来查看哪些能被合并。

anti_join(x, y, by = NULL, ...)
返回y中没有的、x中的行。
※ 可用来查看哪些不能被合并。